TOBB Ekonomi ve Teknoloji Üniversitesi

Elektrik-Elektronik Mühendisliği Bölümü

ELE 362 - Mikrodenetleyiciler ve Uygulamaları

Proje: AVR Tabanlı Oda Ortam İzleme Sistemi

Bahar, 2026

Eda İnan

221201076

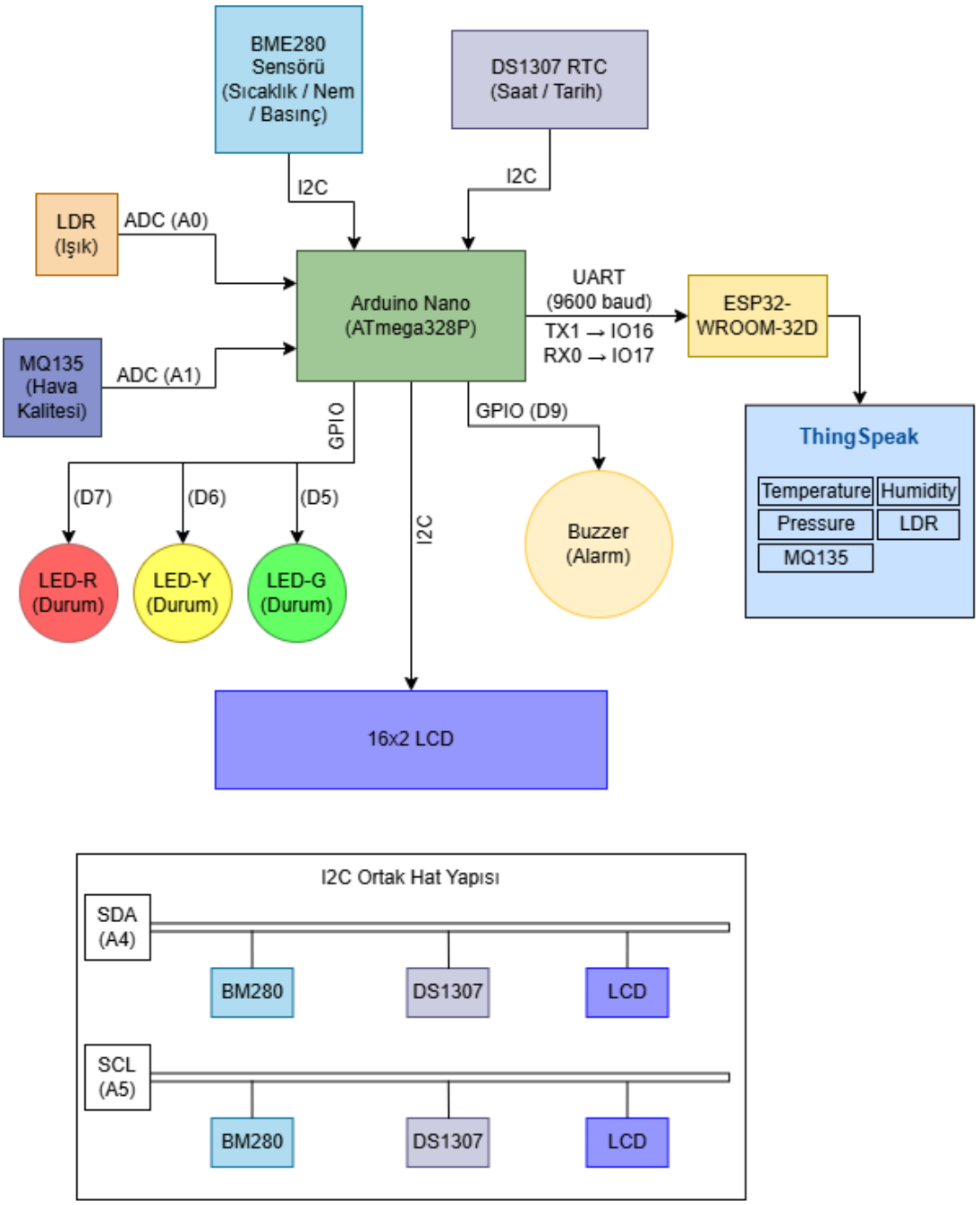
İÇİNDEKİLER

1. GİRİŞ 1
2. SİSTEM MİMARİSİ 3
2.1. Donanım Bileşenleri 3
2.2. İletişim Protokolleri 4
3. YAZILIM TASARIMI 4
3.1. Modül Yapısı 5
3.2. I2C Sürücüsü 6
3.3. BME280 Sürücüsü 7
3.4. DS1307 RTC Sürücüsü 8
3.5. ESP32-WROOM-32D ve ThingSpeak Entegrasyonu 9
3.6. Ana Döngü ve Durum Yönetimi 11
4. DEVRE BAĞLANTILARI 13
5. TEST VE SONUÇLAR 14
5.1. I2C Adres Tarama 14
5.2. Sensör Doğrulaması 15
5.3. ESP8266 ve ThingSpeak 15
6. KARŞILAŞILAN SORUNLAR VE ÇÖZÜMLER 17
7. SONUÇ 18
8. EK-A: KAYNAK KOD DOSYALARI 19
adc.h 19
bme280.h 19
config.h 20
esp32/config.h 20
ds1307.h 20
gpio.h 21
i2c.h 21
lcd.h 22
uart.h 22
adc.c 23
bme280.c 23
ds1307.c 26
gpio.c 28
i2c.c 29
lcd.c 30
uart.c 32
main.c 34
9. KAYNAKÇA 39

1. GİRİŞ

Bu projede ATmega328P mikrodenetleyicisi (Arduino Nano) kullanılarak bir oda ortam izleme sistemi tasarlanmış ve gerçekleştirilmiştir. Sistem; sıcaklık, nem, basınç, ışık seviyesi ve hava kalitesi gibi çevresel parametreleri sürekli olarak ölçmektedir. Bu veriler LCD ekran üzerinde kullanıcıya göstermektedir. Belirli eşik değerlerin aşılması durumunda LED ve buzzer ile uyarı verilmekte ve ölçüm sonuçları ESP32-WROOM-32D modülü aracılığıyla ThingSpeak bulut platformuna iletilmektedir. **Şekil 1**'de devrenin bağlantılarının olduğu blok diyagramını bulabilirsiniz.

Proje; donanım tasarımı, register düzeyinde sürücü yazılımı geliştirme ve kablosuz veri iletimi konularında kapsamlı bir uygulama deneyimi sunmaktadır. Atmel Studio 7 geliştirme ortamında, Arduino kütüphanelerinden bağımsız olarak tüm çevre birim sürücüleri (I2C, UART, ADC) register düzeyinde C programlama dili ile yazılmıştır.



Şekil 1. Oda durum izleyicisi devrenin bağlantılarının blok diyagramı

2. SİSTEM MİMARİSİ

Sistem bir merkezi işlem birimi, dört sensör, üç çıkış birimi ve bir tane kablosuz iletişim modülünden oluşmaktadır. Tüm bileşenler Arduino Nano üzerinden yönetilmektedir.

2.1. Donanım Bileşenleri

Sistemde kullanılan donanım bileşenleri ve kullanım amaçları **Tablo 1**'de verilmiştir. (Atmel Corporation, 2018 **[4]**)

Tablo 1. Donanım Bileşenleri

Bileşen	Kullanım Aracı
ATmega328P (Nano)	Ana mikrodenetleyici
BME280	Sıcaklık, nem, basınç ölçümü (I2C - 0x76)
DS1307 RTC Modülü	Gerçek zamanlı saat ve tarih (I2C - 0x68)
LDR (Işık Sensörü)	Ortam ışık seviyesi (ADC - A0)
MQ135 (Gaz Sensörü)	Ham ADC değeri üzerinden göreceli hava kalitesi ölçümü (ADC - A1)
16x2 LCD (I2C)	Ölçüm sonuçlarının gösterimi (I2C - 0x27)
LED (Kırmızı/Sarı/Yeşil)	Ortam durum göstergesi (GPIO - D5/D6/D7)
Buzzer	Sesli uyarı (GPIO - PB1)
ESP32-WROOM-32D	Kablosuz veri iletimi (UART)
AMS1117 - 3V3 LDO	ESP8266 için 3.3V/800mA güç kaynağı
CR2032 Pili	RTC yedek güç kaynağı

2.2. İletişim Protokolleri

Sistemde üç farklı iletişim protokolü kullanılmıştır:

- I2C (TWI): BME280, DS1307 ve LCD aynı SDA (A4) ile SCL (A5) hattını kullanmaktadır. Her cihazın farklı adresi sayesinde tek hat üzerinden çoklu iletişim sağlanmaktadır. Bu iletişim 100kHz hızında çalışmaktadır.
- ADC: LDR ve MQ135 sensörleri analog sinyal üretmektedir. ATmega328P'nin 10-bit ADC birimi ile A0 ve A1 kanallarından okunmaktadır.
- UART: ESP32-WROOM-32D modülü, Arduino IDE ortamında programlanarak WiFi bağlantısı ve ThingSpeak veri iletimi görevini üstlenmiştir. ATmega328P mikrodenetleyicisi ile UART (9600 baud) üzerinden haberleşmektedir. ATmega328P, sensör verilerini UART üzerinden metin formatında ESP32'ye göndermektedir. ESP32 tarafında bu veriler ayrıştırılmakta ve HTTP GET isteği ile ThingSpeak platformuna iletilmektedir.

3. YAZILIM TASARIMI

Yazılım, Atmel Studio 7 ortamında register düzeyinde C dili kullanılarak geliştirilmiştir. Arduino kütüphanelerinden bağımsız tüm sürücüler sıfırdan yazılmıştır. Proje modüler yapıda tasarlanmış ve her çevre birimi için ayrı .h ve .c dosyaları oluşturulmuştur.

Projenin tüm kaynak kodlarına aşağıdaki GitHub deposundan erişilebilir:

https://github.com/inaneda/room-monitor

3.1. Modül Yapısı

Projenin yazılım modül yapısı **Tablo 2**'de özetlenmiştir. "*esp32/config.h*" dosyası WIFI_SSID, WIFI_PASS ve THINGSPEAK_KEY değişkenlerini içermektedir. Bu dosya proje çalıştırılacağı zaman gerçek değerler ile değiştirilmelidir.

Tablo 2. Modül Yapısı

Dosya	İçerik
i2c.c / i2c.h	TWI register tabanlı I2C sürücüsü (START/STOP/READ/WRITE)
bme280.c / bme280.h	BME280 başlatma, kalibrasyon okuma ve ham veri işleme
ds1307.c / ds1307.h	DS1307 saat okuma/yazma, BCD-Decimal dönüşüm
lcd.c / lcd.h	PCF8574 I2C expander üzerinden 4-bit LCD kontrolü
adc.c / adc.h	LDR ve MQ135 için ADC başlatma ve kanal okuma
uart.c / uart.h	UART başlatma, karakter/string/sayı gönderme ve alma
gpio.c / gpio.h	LED ve buzzer GPIO kontrolü
main.c	Ana döngü, ESP32 UART veri gönderimi, 4 sayfalı LCD yönetimi
config.h	Eşik değerleri (sıcaklık, nem, mq135)
esp32/config.h	WiFi kimlik bilgileri, ThingSpeak API anahtarı
esp32_thingspeak.ino	WiFi bağlantısı, UART parse, ThingSpeak HTTP GET

3.2. I2C Sürücüsü

ATmega328P'nin TWI (Two-Wire Interface) donanımı kullanılarak 100 kHz hızında I2C iletişimi gerçekleştirilmiştir. "*i2c_start()*" fonksiyonu hem START condition üretmekte hem de adres ve yön bitini (R/W) göndermektedir. ACK/NACK durumu TWSR register'ından okunarak kontrol edilmektedir.

TWBR değeri F_CPU ve hedef SCL frekansına göre hesaplanmaktadır: TWBR = (F_CPU / SCL_FREQ - 16) / 2. 16 MHz işlemci ve 100 kHz I2C için TWBR = 72 elde edilmektedir.

```
void i2c_init(void){
     TWSR = 0x00; // prescaler = 1
     TWBR = ((F_CPU / 100000UL) - 16) / 2; // scl yaklasik
100kHz
     TWCR = (1 << TWEN); // TWI etkinlestir
}

uint8_t i2c_start(uint8_t addr_rw){
     // START
     TWCR = (1<<TWINT)|(1<<TWSTA)|(1<<TWEN);
     // tamamlana kadar bekle
     while ( !(TWCR & (1<<TWINT)) );
     // durum okuma
     uint8_t s = TWSR & 0xF8;
     if (s != TW_START && s != TW_REP_START) return 1;

     // adres gönder
     TWDR = addr_rw;
     TWCR = (1<<TWINT)|(1<<TWEN);
     // gonderimin tamamlanmasini bekle
     while ( !(TWCR & (1<<TWINT)) );
     // slave ack dondu mu
     s = TWSR & 0xF8;
     if (s != TW_MT_SLA_ACK && s != TW_MR_SLA_ACK) return 1;
     return 0; // donus: 0 = basarili, 1 = hata
}
```

TWINT bayrağı her işlem sonunda donanım tarafından temizlenmektedir. Kod bu bayrağın tekrar set edilmesini bekleyerek senkronizasyonu sağlamaktadır. TW_MT_SLA_ACK (0x18) ve TW_MR_SLA_ACK (0x40) durum

kodları sırasıyla yazma ve okuma modunda adres ACK'yi ifade etmektedir.

3.3. BME280 Sürücüsü

BME280 sensörü I2C adresi 0x76 ile iletişim kurmaktadır. Başlatma aşamasında chip ID (0x60) doğrulanmakta, oversampling ayarları yapılmakta ve fabrika kalibrasyon katsayıları (dig_T1-T3, dig_P1-P9, dig_H1-H6) okunmaktadır. Ham ADC verileri Bosch'un referans kalibrasyon algoritması (Bosch Sensortec, 2023 **[1]**) ile gerçek fiziksel değerlere dönüştürülmektedir.

```
// sensoru baslatma
uint8_t bme280_init(void) {
     uint8_t id = bme280_read_reg(0xD0);
     if (id != 0x60) return 0;  // Sensor bulunamadi

     bme280_write_reg(0xF2, 0x01);  // humidity oversampling x1
     bme280_write_reg(0xF4, 0x27);  // temp+pressure
oversampling x1, normal mode
     bme280_write_reg(0xF5, 0xA0);  // standby 1000ms

     bme280_read_calibration();
     _delay_ms(100); // kisa bekleme
     return 1;
}

// ham verileri okuma:
// sicaklik
int32_t adc_T = ((int32_t)bme280_read_reg(0xFA) << 12) |
((int32_t)bme280_read_reg(0xFB) << 4) | (bme280_read_reg(0xFC) >>
4);
// basinc
int32_t adc_P = ((int32_t)bme280_read_reg(0xF7) << 12) |
((int32_t)bme280_read_reg(0xF8) << 4) | (bme280_read_reg(0xF9) >>
4);
// nem
int32_t adc_H = ((int32_t)bme280_read_reg(0xFD) << 8) |
bme280_read_reg(0xFE);

// sicaklik hesabi (Bosch referans algoritmasi)
int32_t var1 = ((((adc_T >> 3) - ((int32_t)dig_T1 << 1))) *
dig_T2) >> 11;
     int32_t var2 = (((((adc_T >> 4) - (int32_t)dig_T1) *
((adc_T >> 4) - (int32_t)dig_T1)) >> 12) * dig_T3) >> 14;
     t_fine = var1 + var2;
```

```
      data.temperature = (float)((t_fine * 5 + 128) >> 8) /
100.0f;
```

t_fine ara değişkeni sıcaklığa bağlı bir düzeltme faktörü olup basınç ve nem hesaplarında da kullanılmaktadır. Bu nedenle sıcaklık her zaman önce hesaplanmalıdır.

3.4. DS1307 RTC Sürücüsü

DS1307 gerçek zaman saati entegresi (Maxim Integrated, 2015 **[2]**) I2C adresi 0x68 üzerinden erişilmektedir. Çip, BCD (Binary Coded Decimal) formatında zaman verisi saklamaktadır. Register 0x00'daki CH (Clock Halt) biti sıfırlanarak osilatör etkinleştirilmektedir. Zaman okuma ve yazma işlemlerinde BCD-decimal dönüşüm fonksiyonları kullanılmaktadır. CR2032 yedek pili sayesinde sistem kapalıyken de saat çalışmaya devam etmektedir.

```
// BCD'den decimal'e cevirme
static uint8_t bcd2dec(uint8_t bcd){
      return ((bcd >> 4) * 10) + (bcd & 0x0F);
}

// decimal'den BCD'ye cevirme
static uint8_t dec2bcd(uint8_t dec){
      return ((dec / 10) << 4) | (dec % 10);
}

void ds1307_set_time(DS1307_Time t){
    // CH bitini temizle
    i2c_start(DS1307_ADDR << 1);
    i2c_write(0x00);     // saniye reg adresi
    i2c_write(0x00);     // saniye=0, CH=0
    i2c_stop();

    _delay_ms(10);

    // tum zamani yaz
    i2c_start(DS1307_ADDR << 1);
    i2c_write(0x00);     // reg pointer = 0x00
    i2c_write(dec2bcd(t.second) & 0x7F); // CH bitini sifirlama
    i2c_write(dec2bcd(t.minute));
      // saat reg.i:
      // bit6 = 12/24 saat modu (0 = 24 saat)
    i2c_write(dec2bcd(t.hour) & 0x3F);
    i2c_write(0x01);     // haftanin gunu (1-7 arasi)
    i2c_write(dec2bcd(t.day));
```

```
    i2c_write(dec2bcd(t.month));
    i2c_write(dec2bcd(t.year));
    i2c_stop();
}


DS1307_Time ds1307_get_time(void){
     DS1307_Time t = {0, 0, 0, 0, 0, 0};

     // register pointer'ı 0x00'a set et
     i2c_start(DS1307_ADDR << 1);
     i2c_write(0x00);
     i2c_stop();
     _delay_us(100);

     // okuma baslatma
     i2c_start((DS1307_ADDR << 1) | 1);
     uint8_t raw_sec   = i2c_read_ack();
     uint8_t raw_min   = i2c_read_ack();
     uint8_t raw_hour  = i2c_read_ack();
     i2c_read_ack();   // haftanin gunu yok
     uint8_t raw_day   = i2c_read_ack();
     uint8_t raw_month = i2c_read_ack();
     uint8_t raw_year  = i2c_read_nack();
     i2c_stop();

     t.second = bcd2dec(raw_sec & 0x7F); // CH bitini maskele
     t.minute = bcd2dec(raw_min);
     t.hour   = bcd2dec(raw_hour & 0x3F);
     t.day    = bcd2dec(raw_day);
     t.month  = bcd2dec(raw_month);
     t.year   = bcd2dec(raw_year);

     return t;
}
```

Okuma işleminde önce WRITE moduyla register pointer 0x00'a çekilmekte, ardından READ moduyla 7 byte sırayla okunmaktadır. Son byte'tan sonra NACK gönderilerek okuma sonlandırılmaktadır.

3.5. ESP32-WROOM-32D ve ThingSpeak Entegrasyonu

Geliştirme sürecinde önce ESP8266 WiFi modülü (Espressif Systems, 2023 **[3]**), Arduino Nano'nun donanımsal UART hattı (D0/D1) üzerinden AT komut seti ile yönetilmesi denenmiştir. Bu modül 3.3V ile beslenmesi gerektiğinden Arduino'nun 5V çıkışından AMS1117-3.3 LDO regülatörü aracılığıyla 3.3V/800mA

elde edilmiştir. Modül AT komutlarına UART üzerinden cevap vermiş ancak bu yanıtlar anlamlı gelmemiş ve modül WiFi ağına bağlantı sağlamamıştır. Farklı baud rate değerleri denenmesine ve firmware güncellemesine rağmen sorun çözülememiştir. Bu nedenle ESP32-WROOM-32D modülüne (Espressif Systems, 2023 **[7]**) geçilmiştir.

ESP32 modülü USB kablosu ile doğrudan beslenmiştir. AMS1117-3.3 LDO regülatörü yalnızca ESP8266 denemesi sırasında kullanılmıştır.

ESP32, Arduino IDE ortamında WiFi.h ve HTTPClient.h kütüphaneleri **[10]** kullanılarak ayrıca programlanmıştır. ATmega328P sensör verilerini UART üzerinden 9600 baud hızında T:24,H:51,P:1008,L:300,G:420 formatında ESP32'ye iletmektedir. ESP32 bu veriyi "*indexOf*" ve "*toInt*" fonksiyonları ile ayrıştırarak HTTP GET isteğiyle ThingSpeak'e göndermektedir. Bağlantı kesilmesi durumunda ESP32, "*ensureWifi*" fonksiyonu aracılığıyla WiFi ağına otomatik olarak yeniden bağlanmaktadır.

UART sinyal hattında Arduino'nun TX pini (5V) ile ESP32 RX pini GPIO16 (3.3V toleranslı) arasına gerilim bölücü (1kΩ + 2kΩ) eklenmiştir. Böylelikle haberleşme için gereken gerilim 5V'tan 3.3V'a düşürülmüştür. ESP32'nin TX pini GPIO17, Arduino'nun D0 (RX) pinine bağlanmaktadır. Yaklaşık 16 saniyede bir sensör verileri ThingSpeak'e (ThingSpeak **[5]**) iletilmektedir.

```
/* ESP32'ye UART uzerinden veri gonderme */
void esp32_send_data(uint16_t temp, uint16_t hum, uint16_t pres,
uint16_t ldr, uint16_t mq){
      // veri formati: T:24,H:51,P:1008,L:300,G:420
      uart_send_string("T:");
      uart_send_uint16(temp);
      uart_send_string(",H:");
      uart_send_uint16(hum);
      uart_send_string(",P:");
      uart_send_uint16(pres);
      uart_send_string(",L:");
      uart_send_uint16(ldr);
      uart_send_string(",G:");
      uart_send_uint16(mq);
      uart_send_string("\r\n");
}
```

Basınç değeri ATmega tarafında uint16_t dönüşümü ile tam sayıya çevrilerek gönderilmektedir.

3.6. Ana Döngü ve Durum Yönetimi

Ana döngü sayfa bazlı çalışmaktadır. Her sayfa 4 saniye (PAGE_HOLD_MS=4000) ekranda kalır ve bu süre içinde sensörler 250ms aralıklarla (PAGE_REFRESH_MS = 250) okunur. Toplamda 4 sayfa bulunmaktadır: 1) Tarih ve saat, 2) Sıcaklık, nem ve saat, 3) Işık ve hava kalitesi, 4) Durum ve saat.

Durum mantığı şu öncelik sırasına göre çalışmaktadır: MQ135 eşik değeri aşılırsa kırmızı LED ve buzzer devreye girmektedir. Sıcaklık veya nem sınır dışına çıkarsa sarı LED yanmaktadır. Her iki koşul da normal aralıkta ise yeşil LED yanmaktadır.

```
int main(void) {
     gpio_init();
     adc_init();
     i2c_init();
     lcd_init();
     uart_init(UBRR_VAL);
     _delay_ms(500);

     // DS1307 ilk kurulum: bir kere yapildiktan sonra yorum
satiri yapilir
                                   // sn:dk:sa:gun:ay:yil
     //DS1307_Time init_time = {0, 45, 16, 14, 4, 26};
     //ds1307_set_time(init_time);

     // BME280 kontrol
     if (!bme280_init()) {
          lcd_clear();
          lcd_set_cursor(0, 0);
          lcd_print("BME280 HATA!");
          while(1);
     }

     uint8_t current_page = 0;
     uint8_t send_counter = 0;

     // sistem baslangici tarih ve saat ile
     DS1307_Time t_init = ds1307_get_time();
     lcd_clear();
     lcd_set_cursor(0, 0);
     lcd_print_uint16(t_init.day);   lcd_print("/");
     lcd_print_uint16(t_init.month); lcd_print("/");
     lcd_print_uint16(t_init.year);

     lcd_set_cursor(1, 0);
     lcd_print_uint16(t_init.hour);   lcd_print(":");
     lcd_print_uint16(t_init.minute); lcd_print(":");
     lcd_print_uint16(t_init.second);
     _delay_ms(4000);


     while(1){
          uint16_t elapsed = 0;

          while (elapsed < PAGE_HOLD_MS) {
               // sensor okuma
               BME280_Data bme = bme280_read();
               uint16_t ldr = adc_read(ADC_LDR);
               uint16_t mq  = adc_read(ADC_MQ135);
               DS1307_Time t = ds1307_get_time();

               // lcd guncelleme
               lcd_show_page(current_page, &bme, ldr, mq, &t);
```

```
                update_status(&bme, mq);

                // tekrar guncelleme (gercek zamanli)
                _delay_ms(PAGE_REFRESH_MS);
                elapsed += PAGE_REFRESH_MS;
            }

            // sayfa degistirme
            current_page++;
            if (current_page >= PAGE_COUNT) {
                current_page = 0;
            }

            // yaklasik 16 saniyede bir gonderim
            send_counter++;
            if (send_counter >= 4) {
                BME280_Data bme = bme280_read();
                uint16_t ldr = adc_read(ADC_LDR);
                uint16_t mq  = adc_read(ADC_MQ135);

                esp32_send_data(
                (uint16_t)bme.temperature,
                (uint16_t)bme.humidity,
                (uint16_t)bme.pressure,
                ldr,
                mq
                );
                send_counter = 0;
            }
        }
}
```

4. DEVRE BAĞLANTILARI

Tüm bileşenlerin Arduino Nano'ya bağlantı bilgileri **Tablo 3**'te verilmiştir.

Tablo 3. Devre Bağlantıları

Bileşen - Pin	Arduino Nano Bağlantısı
BME280 SDA	A4 (I2C SDA, ortak hat)
BME280 SCL	A5 (I2C SCL, ortak hat)
BME280 VCC	3.3V
BME280 GND	GND (ortak)
DS1307 SDA	A4 (I2C SDA, ortak hat)

DS1307 SCL	A5 (I2C SCL, ortak hat)
DS1307 VCC	5V (ortak)
DS1307 GND	GND (ortak)
LCD SDA	A4 (I2C SDA, ortak hat)
LCD SCL	A5 (I2C SCL, ortak hat)
LCD VCC	5V (ortak)
LDR	A0 (ADC Kanal 0)
MQ135	A1 (ADC Kanal 1)
LED Kırmızı	D5 (GPIO)
LED Sarı	D6 (GPIO)
LED Yeşil	D7 (GPIO)
Buzzer	PB1 / D9 (GPIO)
ESP32 GND	GND (ortak)
ESP32 TX (GPIO17)	D0 (UART RX)
ESP32 RX (GPIO16)	D1 (UART TX) (gerilim bölücü (1kΩ+2kΩ))

5. TEST VE SONUÇLAR

Sistem, modüler olarak test edilmiştir. Her sürücü bağımsız biçimde doğrulandıktan sonra entegrasyon gerçekleştirilmiştir.

5.1. I2C Adres Tarama

Geliştirme sürecinde yazılan I2C scanner kodu ile tüm I2C cihazları doğrulanmıştır. I2C tarama sonucunda tespit edilen cihaz adresleri **Tablo 4**'te listelenmiştir.

Tablo 4. I2C Adres Tarama Sonuçları

Adres (Decimal)	Cihaz
39 (0x27)	LCD I2C Genişletici (PCF8574)
80 (0x50)	DS1307 Dahili EEPROM
104 (0x68)	DS1307 RTC
118 (0x76)	BME280 Sensörü

5.2. Sensör Doğrulaması

- BME280: Sıcaklık ölçümleri referans termometre ile karşılaştırılmış, ±1°C hassasiyet gözlemlenmiştir.
- DS1307: Saat değerleri 24 saatlik gözlem boyunca doğru çalışmıştır. CR2032 pili sayesinde güç kesintilerinde zaman bilgisi korunmuştur.
- LDR: Aydınlık ve karanlık ortam arasında belirgin ADC değer farkı gözlemlenmiştir.
- MQ135: Isınma süresinin ardından sensör stabil değerler üretmiştir.

5.3. ESP8266 ve ThingSpeak

ESP32-WROOM-32D modülü USB kablosu ile beslenmiştir. ATmega328P'den UART üzerinden gelen veriler başarıyla ayrıştırılmış ve ThingSpeak'e iletilmiştir. Yaklaşık 16 saniyede bir gerçekleştirilen güncelleme işlemi ile sıcaklık, nem, basınç, ışık ve hava kalitesi verileri bulut platformuna aktarılmaktadır. Sisteme ait ThingSpeak kanal görünümü **Şekil 2**'de verilmiştir.

Şekil 2. ThingSpeak platformunda görüntülenen sensör verileri.

Alan-1: Sıcaklık (°C), Alan-2: Nem (%), Alan-3: Basınç (hPa),

Alan-4: Işık (ADC), Alan-5: Hava Kalitesi (ADC).

6. KARŞILAŞILAN SORUNLAR VE ÇÖZÜMLER

Geliştirme sürecinde karşılaşılan sorunlar ve uygulanan çözümler **Tablo 5**'te özetlenmiştir.

Tablo 5. Karşılaşılan Sorunlar ve Çözümler

Sorun	Çözüm
DS1307 I2C'de görünmüyordu	DIP8 breadboard bağlantısı yerine hazır Tiny RTC modülüne geçildi; pull-up dirençler modül üzerinde mevcuttu
DS1307 CH biti 1 olarak geldi	set_time çağrısında register 0x00 bit 7 açıkça 0'a çekildi
i2c_start() adres almıyordu	Fonksiyon imzası i2c_start(uint8_t addr_rw) olarak güncellendi, adres ve R/W biti fonksiyon içinde gönderilir hale getirildi
ESP8266 Arduino 3.3V piniyle çalışmıyordu	AMS1117-3.3 LDO regülatörü eklendi (800mA çıkış kapasitesi)
ESP8266 RX pinine 5V zarar verebilirdi	1kΩ + 2kΩ gerilim bölücü ile 5V'tan 3.3V sinyal dönüşümü yapıldı
UART baud rate uyumsuzluğu	115200 baud'da ATmega328P'de %4.5 hata oluşuyordu; 9600 baud'a geçildi
BME280 nem hesabı yanlıştı	Bosch referans kodundaki h değişkeni doğru hesaplanıp döndürüldü
ESP8266 ile iletişim doğru sağlanamadı	Farklı baud rate'ler denendi ama gelen veri anlamlı olmadı ve WiFi bağlantısı sağlanamadı bu nedenle ESP32'ye geçildi

7. SONUÇ

Bu projede ATmega328P mikrodenetleyicisi kullanılarak çok sensörlü bir oda ortam izleme sistemi başarıyla tasarlanmıştır. I2C, ADC ve UART iletişim protokolleri register düzeyinde C ile yazılmıştır. Kodlar Arduino kütüphanelerine bağımlılık olmadan tüm sürücüler sıfırdan yazılmıştır. (ESP32 hariç)

Sistem; BME280, DS1307, LDR ve MQ135 sensörlerinden gerçek zamanlı veri toplamakta, LCD üzerinde göstermekte, durum LED'leri ve buzzer ile uyarı vermekte ve ESP32-WROOM-32D aracılığıyla ThingSpeak platformuna iletmektedir. Geliştirme sürecinde karşılaşılan donanım sorunları (güç, gerilim uyumsuzluğu, breadboard temas direnci, ESP8266 bağlantı sorunları) sistematik olarak çözülmüştür.

8. EK-A: KAYNAK KOD DOSYALARI

adc.h

```
#ifndef ADC_H_
#define ADC_H_

#include <avr/io.h>

// kanal tanimlari
#define ADC_LDR     0   // A0
#define ADC_MQ135   1   // A1

// fonksiyonlar
void adc_init(void);
uint16_t adc_read(uint8_t channel);

#endif /* ADC_H_ */
```

bme280.h

```
#ifndef BME280_H_
#define BME280_H_

#include <avr/io.h>

#define BME280_ADDR 0x76  // SDO GND'ye bagli -> 0x76, VCC'ye
bagli olsaydi -> 0x77

typedef struct {
      float temperature;
      float humidity;
      float pressure;
} BME280_Data;

uint8_t bme280_init(void);
BME280_Data bme280_read(void);

#endif /* BME280_H_ */
```

config.h

```
#ifndef CONFIG_H_
#define CONFIG_H_

#define TEMP_MIN   18
#define TEMP_MAX   28
#define HUM_MIN    30
#define HUM_MAX    70
#define MQ135_THRESHOLD  650

#endif /* CONFIG_H_ */
```

esp32/config.h

```
#ifndef CONFIG_H
#define CONFIG_H

#define WIFI_SSID          "your_wifi_name"
#define WIFI_PASS          "your_wifi_password"
#define THINGSPEAK_KEY     "your_api_key"

#endif
```

ds1307.h

```
#ifndef DS1307_H_
#define DS1307_H_

#include <avr/io.h>

#define DS1307_ADDR 0x68

typedef struct {
     uint8_t second;
     uint8_t minute;
     uint8_t hour;
     uint8_t day;
     uint8_t month;
     uint8_t year;
} DS1307_Time;

void ds1307_set_time(DS1307_Time t);
DS1307_Time ds1307_get_time(void);

#endif /* DS1307_H_ */
```

gpio.h

```
#ifndef GPIO_H_
#define GPIO_H_

#include <avr/io.h>

// LED pin tanimlari (PORTD)
#define LED_RED_PIN    PD5
#define LED_YELLOW_PIN PD6
#define LED_GREEN_PIN  PD7

// Buzzer pin tanimi (PORTB)
#define BUZZER_PIN     PB1

// Fonksiyon prototipleri
void gpio_init(void);
void led_set(uint8_t red, uint8_t yellow, uint8_t green);
void buzzer_on(void);
void buzzer_off(void);

#endif /* GPIO_H_ */
```

i2c.h

```
#ifndef I2C_H_
#define I2C_H_

#include <avr/io.h>

#define F_CPU 16000000UL

void i2c_init(void);
uint8_t i2c_start(uint8_t addr_rw);
void i2c_stop(void);
uint8_t i2c_write(uint8_t data);
uint8_t i2c_read_ack(void);
uint8_t i2c_read_nack(void);

#endif
```

lcd.h

```
#ifndef LCD_H_
#define LCD_H_

#include <avr/io.h>

#define LCD_ADDR 0x27  // I2C adresi (0x3F de olabilir)

void lcd_send_char(char c);

void lcd_init(void);
void lcd_clear(void);
void lcd_set_cursor(uint8_t row, uint8_t col);
void lcd_print(const char *str);
void lcd_print_uint16(uint16_t val);

#endif /* LCD_H_ */
```

uart.h

```
#ifndef UART_H_
#define UART_H_

#include <avr/io.h>
#include <stdint.h>

uint8_t uart_uint16_to_str(uint16_t val, char *buf);
uint8_t uart_receive_char(void);

void uart_init(uint16_t ubrr);
void uart_send_char(char c);
void uart_send_string(const char *str);
void uart_send_uint16(uint16_t val);

uint8_t uart_read_response(char *buf, uint8_t max_len, uint16_t
wait_ms);
uint8_t uart_contains(const char *buf, const char *key);

#endif /* UART_H_ */
```

adc.c

```
#include "adc.h"

void adc_init(void){
	// VCC referans voltaji
	ADMUX = (1 << REFS0);
	// ADC'yi etkinlestirme, prescaler 128 (16MHz/128 = 125kHz)
	ADCSRA = (1 << ADEN) | (1 << ADPS2) | (1 << ADPS1) | (1 << 
ADPS0);
}

uint16_t adc_read(uint8_t channel){
	// kanal secimi
	ADMUX = (ADMUX & 0xF0) | (channel & 0x0F);
	// donusumu baslat
	ADCSRA |= (1 << ADSC);
	// donusumun bitmesini bekle
	while (ADCSRA & (1 << ADSC));
	return ADC;
}
```

bme280.c

```
#include "bme280.h"
#include "i2c.h"
#include <util/delay.h>

// kalibrasyon verileri
static uint16_t dig_T1;
static int16_t  dig_T2, dig_T3;
static uint16_t dig_P1;
static int16_t  dig_P2, dig_P3, dig_P4, dig_P5, dig_P6, dig_P7, 
dig_P8, dig_P9;
static uint8_t  dig_H1, dig_H3;
static int16_t  dig_H2, dig_H4, dig_H5;
static int8_t   dig_H6;
static int32_t  t_fine; // bosch algoritmasinda ara sicaklik 
duzeltme degeri

// tek register okuma
static uint8_t bme280_read_reg(uint8_t reg) {
	i2c_start(BME280_ADDR << 1);
	i2c_write(reg);
	i2c_start((BME280_ADDR << 1) | 1);
	uint8_t val = i2c_read_nack();
	i2c_stop();
	return val;
}

// tek register yazma
```

```
static void bme280_write_reg(uint8_t reg, uint8_t val) {
      i2c_start(BME280_ADDR << 1);
      i2c_write(reg);
      i2c_write(val);
      i2c_stop();
}

static void bme280_read_calibration(void) {
      dig_T1 = (bme280_read_reg(0x89) << 8) |
bme280_read_reg(0x88);
      dig_T2 = (bme280_read_reg(0x8B) << 8) |
bme280_read_reg(0x8A);
      dig_T3 = (bme280_read_reg(0x8D) << 8) |
bme280_read_reg(0x8C);
      dig_P1 = (bme280_read_reg(0x8F) << 8) |
bme280_read_reg(0x8E);
      dig_P2 = (bme280_read_reg(0x91) << 8) |
bme280_read_reg(0x90);
      dig_P3 = (bme280_read_reg(0x93) << 8) |
bme280_read_reg(0x92);
      dig_P4 = (bme280_read_reg(0x95) << 8) |
bme280_read_reg(0x94);
      dig_P5 = (bme280_read_reg(0x97) << 8) |
bme280_read_reg(0x96);
      dig_P6 = (bme280_read_reg(0x99) << 8) |
bme280_read_reg(0x98);
      dig_P7 = (bme280_read_reg(0x9B) << 8) |
bme280_read_reg(0x9A);
      dig_P8 = (bme280_read_reg(0x9D) << 8) |
bme280_read_reg(0x9C);
      dig_P9 = (bme280_read_reg(0x9F) << 8) |
bme280_read_reg(0x9E);
      dig_H1 = bme280_read_reg(0xA1);
      dig_H2 = (bme280_read_reg(0xE2) << 8) |
bme280_read_reg(0xE1);
      dig_H3 = bme280_read_reg(0xE3);
      dig_H4 = ((int16_t)(int8_t)bme280_read_reg(0xE4) << 4) |
(bme280_read_reg(0xE5) & 0x0F);
      dig_H5 = ((int16_t)(int8_t)bme280_read_reg(0xE6) << 4) |
(bme280_read_reg(0xE5) >> 4);
      dig_H6 = (int8_t)bme280_read_reg(0xE7);
}

// sensoru baslatma
uint8_t bme280_init(void) {
      uint8_t id = bme280_read_reg(0xD0);
      if (id != 0x60) return 0;  // Sensor bulunamadi

      bme280_write_reg(0xF2, 0x01);  // humidity oversampling x1
      bme280_write_reg(0xF4, 0x27);  // temp+pressure
oversampling x1, normal mode
      bme280_write_reg(0xF5, 0xA0);  // standby 1000ms

      bme280_read_calibration();
      _delay_ms(100); // kisa bekleme
```

```
	return 1;
}

// sicaklik, basinc ve nem okuma
BME280_Data bme280_read(void) {
	BME280_Data data = {0, 0, 0};

	// ham verileri okuma:
	// sicaklik
	int32_t adc_T = ((int32_t)bme280_read_reg(0xFA) << 12) |
((int32_t)bme280_read_reg(0xFB) << 4) | (bme280_read_reg(0xFC) >>
4);
	// basinc
	int32_t adc_P = ((int32_t)bme280_read_reg(0xF7) << 12) |
((int32_t)bme280_read_reg(0xF8) << 4) | (bme280_read_reg(0xF9) >>
4);
	// nem
	int32_t adc_H = ((int32_t)bme280_read_reg(0xFD) << 8) |
bme280_read_reg(0xFE);

	// sicaklik hesabi (Bosch referans algoritmasi)
	int32_t var1 = ((((adc_T >> 3) - ((int32_t)dig_T1 << 1))) *
dig_T2) >> 11;
	int32_t var2 = (((((adc_T >> 4) - (int32_t)dig_T1) *
((adc_T >> 4) - (int32_t)dig_T1)) >> 12) * dig_T3) >> 14;
	t_fine = var1 + var2;
	data.temperature = (float)((t_fine * 5 + 128) >> 8) /
100.0f;

	// basinc hesabi (Bosch referans algoritmasi)
	int64_t p_var1 = (int64_t)t_fine - 128000;
	int64_t p_var2 = p_var1 * p_var1 * (int64_t)dig_P6;
	p_var2 = p_var2 + ((p_var1 * (int64_t)dig_P5) << 17);
	p_var2 = p_var2 + (((int64_t)dig_P4) << 35);
	p_var1 = ((p_var1 * p_var1 * (int64_t)dig_P3) >> 8) +
((p_var1 * (int64_t)dig_P2) << 12);
	p_var1 = (((int64_t)1 << 47) + p_var1) * (int64_t)dig_P1 >>
33;

	if (p_var1 != 0) {
		int64_t p = 1048576 - adc_P;
		p = (((p << 31) - p_var2) * 3125) / p_var1;
		p_var1 = ((int64_t)dig_P9 * (p >> 13) * (p >> 13)) >>
25;
		p_var2 = ((int64_t)dig_P8 * p) >> 19;
		p = ((p + p_var1 + p_var2) >> 8) + ((int64_t)dig_P7 <<
4);
		data.pressure = (float)p / 25600.0f;
	}

	// nem hesabi (Bosch referans algoritmasi)
	int32_t h = t_fine - 76800;
	h = (((adc_H << 14) - ((int32_t)dig_H4 << 20) -
((int32_t)dig_H5 * h)) + 16384) >> 15;
```

```
     h = h * (((((((int32_t)dig_H6 * h) >> 10) *
(((int32_t)dig_H3 * h) >> 11) + 32768)) >> 10) + 2097152);
     h = (h * (int32_t)dig_H2 + 8192) >> 14;
     h = h - (((((h >> 15) * (h >> 15)) >> 7) * (int32_t)dig_H1)
>> 4);
     // nem degerini 0 ile 100% arasinda sinirlama
     h = h < 0 ? 0 : h;
     h = h > 419430400 ? 419430400 : h;

     //data.humidity = (float)(h >> 12) / 1024.0f;

     data.humidity = (float)(t_fine - 76800) / 1000.0f;

     return data;
}
```

ds1307.c

```
#include <util/delay.h>
#include "ds1307.h"
#include "i2c.h"
// #include "lcd.h"

// BCD'den decimal'e cevirme
static uint8_t bcd2dec(uint8_t bcd){
     return ((bcd >> 4) * 10) + (bcd & 0x0F);
}

// decimal'den BCD'ye cevirme
static uint8_t dec2bcd(uint8_t dec){
     return ((dec / 10) << 4) | (dec % 10);
}

void ds1307_set_time(DS1307_Time t){
    // CH bitini temizle
    i2c_start(DS1307_ADDR << 1);
    i2c_write(0x00); // saniye reg adresi
    i2c_write(0x00); // saniye=0, CH=0
    i2c_stop();

    _delay_ms(10);

    // tum zamani yaz
    i2c_start(DS1307_ADDR << 1);
    i2c_write(0x00); // reg pointer = 0x00
    i2c_write(dec2bcd(t.second) & 0x7F); // CH bitini sifirlama
    i2c_write(dec2bcd(t.minute));
     // saat reg.i:
     // bit6 = 12/24 saat modu (0 = 24 saat)
    i2c_write(dec2bcd(t.hour) & 0x3F);
    i2c_write(0x01); // haftanin gunu (1-7 arasi)
    i2c_write(dec2bcd(t.day));
```

```
    i2c_write(dec2bcd(t.month));
    i2c_write(dec2bcd(t.year));
    i2c_stop();
}

DS1307_Time ds1307_get_time(void){
     DS1307_Time t = {0, 0, 0, 0, 0, 0};

     // register pointer'ı 0x00'a set et
     i2c_start(DS1307_ADDR << 1);
     i2c_write(0x00);
     i2c_stop();
     _delay_us(100);

     // okuma baslatma
     i2c_start((DS1307_ADDR << 1) | 1);
     uint8_t raw_sec   = i2c_read_ack();
     uint8_t raw_min   = i2c_read_ack();
     uint8_t raw_hour  = i2c_read_ack();
     i2c_read_ack();   // haftanin gunu yok
     uint8_t raw_day   = i2c_read_ack();
     uint8_t raw_month = i2c_read_ack();
     uint8_t raw_year  = i2c_read_nack();
     i2c_stop();

     t.second = bcd2dec(raw_sec & 0x7F); // CH bitini maskele
     t.minute = bcd2dec(raw_min);
     t.hour   = bcd2dec(raw_hour & 0x3F);
     t.day    = bcd2dec(raw_day);
     t.month  = bcd2dec(raw_month);
     t.year   = bcd2dec(raw_year);

     return t;
}
```

gpio.c

```
#include "gpio.h"
#include <util/delay.h>

void gpio_init(void){
	// LED pinleri cikis
	DDRD |= (1 << LED_RED_PIN) | (1 << LED_YELLOW_PIN) | (1 <<
LED_GREEN_PIN);
	// Buzzer pini cikis
	DDRB |= (1 << BUZZER_PIN);
	// default olarak kapalilar
	PORTD &= ~((1 << LED_RED_PIN) | (1 << LED_YELLOW_PIN) | (1
<< LED_GREEN_PIN));
	PORTB &= ~(1 << BUZZER_PIN);
}

// LED'lerin durumunu ayarlama
// parametreler: 1 = acik, 0 = kapali
void led_set(uint8_t red, uint8_t yellow, uint8_t green){
	// kirmizi led
	if (red)    PORTD |= (1 << LED_RED_PIN);
	else        PORTD &= ~(1 << LED_RED_PIN);
	// sari led
	if (yellow) PORTD |= (1 << LED_YELLOW_PIN);
	else        PORTD &= ~(1 << LED_YELLOW_PIN);
	// yesil led
	if (green)  PORTD |= (1 << LED_GREEN_PIN);
	else        PORTD &= ~(1 << LED_GREEN_PIN);
}

void buzzer_on(void){
	for (int i = 0; i < 1000; i++){
		PORTB |=  (1 << BUZZER_PIN);
		_delay_us(500);   // 1kHz kare dalga
		PORTB &= ~(1 << BUZZER_PIN);
		_delay_us(500);
	}
}
void buzzer_off(void){
	PORTB &= ~(1 << BUZZER_PIN);
}
```

i2c.c

```
#include "i2c.h"
#include <util/twi.h>

void i2c_init(void){
	TWSR = 0x00; // prescaler = 1
	TWBR = ((F_CPU / 100000UL) - 16) / 2; // scl yaklasik
100kHz
	TWCR = (1 << TWEN); // TWI etkinlestir
}

uint8_t i2c_start(uint8_t addr_rw){
	// START
	TWCR = (1<<TWINT)|(1<<TWSTA)|(1<<TWEN);
	// tamamlana kadar bekle
	while ( !(TWCR & (1<<TWINT)) );
	// durum okuma
	uint8_t s = TWSR & 0xF8;
	if (s != TW_START && s != TW_REP_START) return 1;

	// adres gönder
	TWDR = addr_rw;
	TWCR = (1<<TWINT)|(1<<TWEN);
	// gonderimin tamamlanmasini bekle
	while ( !(TWCR & (1<<TWINT)) );
	// slave ack dondu mu
	s = TWSR & 0xF8;
	if (s != TW_MT_SLA_ACK && s != TW_MR_SLA_ACK) return 1;
	return 0; // donus: 0 = basarili, 1 = hata
}

void i2c_stop(void){
	// STOP
	TWCR = (1<<TWINT)|(1<<TWSTO)|(1<<TWEN);
}

// slave'e 1 byte veri yazma
uint8_t i2c_write(uint8_t data){
	TWDR = data;
	TWCR = (1<<TWINT)|(1<<TWEN);
	while ( !(TWCR & (1<<TWINT)) );
	return (TWSR & 0xF8); // donus: TWI durum kodu
}

// 1 byte veri okuma ve ACK gonderme
uint8_t i2c_read_ack(void) {
	TWCR = (1<<TWINT)|(1<<TWEN)|(1<<TWEA);
	// yazmanin tamamlanmasini bekle
	while ( !(TWCR & (1<<TWINT)) );
	return TWDR;
}

// 1 byte veri okuma ve NACK gonderme
```

```
uint8_t i2c_read_nack(void) {
	TWCR = (1<<TWINT)|(1<<TWEN);
	// okumanin tamamlanmasini bekle
	while ( !(TWCR & (1<<TWINT)) );
	return TWDR;
}
```

lcd.c

```
#include "lcd.h"
#include "i2c.h"
#include <util/delay.h>

// PCF8574 I2C expander uzerindeki LCD kontrol bitleri
#define LCD_BACKLIGHT 0x08
#define LCD_EN        0x04
#define LCD_RW        0x02
#define LCD_RS        0x01

// mode = 0 : komut gonder, LCD_RS : veri gonder
static void lcd_send_byte(uint8_t data, uint8_t mode) {
	uint8_t high = (data & 0xF0) | LCD_BACKLIGHT | mode;
	uint8_t low  = ((data << 4) & 0xF0) | LCD_BACKLIGHT | mode;

	i2c_start(LCD_ADDR << 1);

	i2c_write(high | LCD_EN); // Enable = 1
	_delay_us(1);
	i2c_write(high & ~LCD_EN); // Enable = 0
	_delay_us(50);

	i2c_write(low | LCD_EN); // Enable = 1
	_delay_us(1);
	i2c_write(low & ~LCD_EN); // Enable = 0
	_delay_us(50);

	i2c_stop();
}

// tek karakter yazdirma
void lcd_send_char(char c) {
	lcd_send_byte((uint8_t)c, LCD_RS);
}

// komut gonderme
static void lcd_cmd(uint8_t cmd) {
	lcd_send_byte(cmd, 0);
}

// veri gonderme
static void lcd_data(uint8_t data) {
	lcd_send_byte(data, LCD_RS);
```

```
}

//START
void lcd_init(void) {
	_delay_ms(50);

	// 4-bit mod
	lcd_cmd(0x33);
	_delay_ms(5);
	lcd_cmd(0x32);
	_delay_ms(1);

	lcd_cmd(0x28);  // 4-bit, 2 satir
	lcd_cmd(0x0C);  // display on, cursor off
	// char yazildikca cursor -> saga
	lcd_cmd(0x06);  // entry mode
	lcd_clear();
}

// ekran temizleme
void lcd_clear(void) {
	lcd_cmd(0x01);
	_delay_ms(2);
}

// cursor -> satira-sutuna tasima
void lcd_set_cursor(uint8_t row, uint8_t col) {
	uint8_t addr = (row == 0) ? 0x00 : 0x40;
	lcd_cmd(0x80 | (addr + col));
}

// string yazdirma
void lcd_print(const char *str) {
	while (*str) {
		lcd_data(*str++);
	}
}

// uint16_t sayi yazdirma
void lcd_print_uint16(uint16_t val) {
	char buf[6];
	// sayi -> char dizisi
	buf[5] = '\0';
	buf[4] = '0' + (val % 10); val /= 10;
	buf[3] = '0' + (val % 10); val /= 10;
	buf[2] = '0' + (val % 10); val /= 10;
	buf[1] = '0' + (val % 10); val /= 10;
	buf[0] = '0' + (val % 10);
	char *p = buf; // gereksiz 0'lari kaldir
	while (*p == '0' && *(p+1) != '\0') p++;
	lcd_print(p);
}
```

uart.c

```
#include <avr/io.h>
#include "uart.h"
#include <util/delay.h>

// uint16_t -> string
uint8_t uart_uint16_to_str(uint16_t val, char *buf){
	uint8_t len = 0;
	char tmp[6];
	uint8_t i = 0;

	if (val == 0){
		buf[0] = '0';
		buf[1] = '\0';
		return 1;
	}
	// sirayi tersleme
	while (val > 0){
		tmp[i++] = '0' + (val % 10);
		val /= 10;
	}
	// dogru siraya cevirme
	for (uint8_t j = 0; j < i; j++) buf[j] = tmp[i-1-j];

	buf[i] = '\0';
	return i; // donus: strinf uzunlugu
}

// START
void uart_init(uint16_t ubrr) {
	UBRR0H = (ubrr >> 8); // baudrate - UBBR_VAL
	UBRR0L = ubrr;
	// alici ve verici etkinlestirme
	UCSR0B = (1 << TXEN0) | (1 << RXEN0);
	// 8 bit, 1 stop bit, no parity
	UCSR0C = (1 << UCSZ01) | (1 << UCSZ00);
}

// tek char alma
uint8_t uart_receive_char(void){
	uint32_t timeout = 50000000UL;

	// veri gelene veya timeout dolana kadar bekle
	while ( !(UCSR0A & (1 << RXC0)) && --timeout);
	if (timeout == 0) return '?'; // timout icin hata
	return UDR0; // donus: gelen char
}

// tek char gonderme
void uart_send_char(char c){
	// reg bosalana kadar bekle
	while ( !(UCSR0A & (1 << UDRE0)) );
	UDR0 = c;
```

```
}

// string atma
void uart_send_string(const char *str){
	while (*str) {
		uart_send_char(*str++);
	}
}

// uin16_t sayi gonderme
void uart_send_uint16(uint16_t val){
	char buf[6];
	buf[5] = '\0';
	buf[4] = '0' + (val % 10); val /= 10;
	buf[3] = '0' + (val % 10); val /= 10;
	buf[2] = '0' + (val % 10); val /= 10;
	buf[1] = '0' + (val % 10); val /= 10;
	buf[0] = '0' + (val % 10);
	// bastaki sifirlar atlansin
	char *p = buf;
	while (*p == '0' && *(p+1) != '\0') p++;
	uart_send_string(p);
}

uint8_t uart_read_response(char *buf, uint8_t max_len, uint16_t
wait_ms){ // max_len: buffer boyutu, wait_ms: toplam bekleme
suresi
	uint8_t i = 0;

	while (wait_ms--) {
		uint16_t inner = 0;

		while (UCSR0A & (1 << RXC0)){ // veri oldugu surece
			char c = UDR0;
			if (i < (max_len - 1)){ // buff tasmasi
				buf[i++] = c;
			}
		}

		_delay_ms(1);

		// kisa bir sure veri geldiyse devam et
		inner++;
		(void)inner;
	}

	buf[i] = '\0';
	return i; // donus: okunan toplam char sayisi
}

// metinde belirli bir anahtar kelime var mi
uint8_t uart_contains(const char *buf, const char *key) {
	uint8_t i, j;

	for (i = 0; buf[i] != '\0'; i++) {
```

```
            j = 0;
            while (key[j] != '\0' && buf[i + j] == key[j]) {
                  j++;
            }
            if (key[j] == '\0') {
                  return 1;
            }
      }
      return 0; // donus: 1=var, 0=yok
}
```

main.c

```
#include <avr/io.h>
#include <util/delay.h>
#include "gpio.h"
#include "adc.h"
#include "i2c.h"
#include "lcd.h"
#include "bme280.h"
#include "uart.h"
#include "config.h"
#include "secrets.h"
#include "ds1307.h"

#define BAUD 9600
#define UBRR_VAL (F_CPU/16/BAUD - 1)

#define PAGE_HOLD_MS 4000 // bir sayfanin lcd'de bulunma suresi
#define PAGE_REFRESH_MS 250 // ayni sayfa icinde veri yenileme
(gercek zamanli veri aktarimi icin)
#define PAGE_COUNT 4

/* ESP32'ye UART uzerinden veri gonderme */
void esp32_send_data(uint16_t temp, uint16_t hum, uint16_t pres,
uint16_t ldr, uint16_t mq){
      // veri formati: T:24,H:51,P:1008,L:300,G:420
      uart_send_string("T:");
      uart_send_uint16(temp);
      uart_send_string(",H:");
      uart_send_uint16(hum);
      uart_send_string(",P:");
      uart_send_uint16(pres);
      uart_send_string(",L:");
      uart_send_uint16(ldr);
      uart_send_string(",G:");
      uart_send_uint16(mq);
      uart_send_string("\r\n");
}

// ortam durumuna gore LED ve buzzer
void update_status(BME280_Data *d, uint16_t mq){
```

```
     // kirmizi: hava kalitesi kotu
     if (mq > MQ135_THRESHOLD){
          led_set(1, 0, 0);
          buzzer_on();
          return;
     }
     // sair: sicaklik veya nem sinir disi
     if (d->temperature > TEMP_MAX || d->temperature < TEMP_MIN
|| d->humidity > HUM_MAX || d->humidity < HUM_MIN){
          led_set(0, 1, 0);
          buzzer_off();
          return;
     }
     // yesil: her sey normal
     led_set(0, 0, 1);
     buzzer_off();
}

// LCD'de tek sayfa sensor verisi yazma
void lcd_show_page(uint8_t page, BME280_Data *d, uint16_t ldr,
uint16_t mq, DS1307_Time *t){

     lcd_clear();
     switch (page){
          case 0:
          // Ekran 1: Tarih + Saat
          lcd_set_cursor(0, 0);
          lcd_print_uint16(t->day);   lcd_print("/");
          lcd_print_uint16(t->month); lcd_print("/");
          lcd_print_uint16(t->year);

          lcd_set_cursor(1, 0);
          lcd_print_uint16(t->hour);   lcd_print(":");
          lcd_print_uint16(t->minute); lcd_print(":");
          lcd_print_uint16(t->second);
          break;

          case 1:
          // Ekran 2: Sicaklik + Nem ; Saat
          lcd_set_cursor(0, 0);
          lcd_print("T:");
          lcd_print_uint16((uint16_t)d->temperature);
          lcd_print("C H:");
          lcd_print_uint16((uint16_t)d->humidity);
          lcd_print("%");

          lcd_set_cursor(1, 0);
          lcd_print_uint16(t->hour);   lcd_print(":");
          lcd_print_uint16(t->minute); lcd_print(":");
          lcd_print_uint16(t->second);
          break;

          case 2:
          // Ekran 3: Isik seviyesi + Hava kalitesi
          lcd_set_cursor(0, 0);
```

```
          lcd_print("Isik:");
          lcd_print_uint16(ldr);

          lcd_set_cursor(1, 0);
          lcd_print("Hava:");
          lcd_print_uint16(mq);
          break;

          case 3:
          // Ekran 4: Sistemin durumu ; Saat
          lcd_set_cursor(0, 0);
          if (mq > MQ135_THRESHOLD){
               lcd_print("ALARM");
          }
          else if (d->temperature > TEMP_MAX || d->temperature <
TEMP_MIN || d->humidity > HUM_MAX     || d->humidity < HUM_MIN){
               lcd_print("UYARI");
          }
          else {
               lcd_print("NORMAL");
          }

          lcd_set_cursor(1, 0);
          lcd_print_uint16(t->hour);   lcd_print(":");
          lcd_print_uint16(t->minute); lcd_print(":");
          lcd_print_uint16(t->second);
          break;

          default:
          lcd_set_cursor(0, 0);
          lcd_print("Sayfa Hatasi");
          break;
     }
}


int main(void) {
     gpio_init();
     adc_init();
     i2c_init();
     lcd_init();
     uart_init(UBRR_VAL);
     _delay_ms(500);

     // DS1307 ilk kurulum: bir kere yapildiktan sonra yorum
satiri yapilir
                                   // sn:dk:sa:gun:ay:yil
     //DS1307_Time init_time = {0, 45, 16, 14, 4, 26};
     //ds1307_set_time(init_time);

     // BME280 kontrol
     if (!bme280_init()) {
          lcd_clear();
          lcd_set_cursor(0, 0);
          lcd_print("BME280 HATA!");
```

```
        while(1);
    }

    uint8_t current_page = 0;
    uint8_t send_counter = 0;

    // sistem baslangici tarih ve saat ile
    DS1307_Time t_init = ds1307_get_time();
    lcd_clear();
    lcd_set_cursor(0, 0);
    lcd_print_uint16(t_init.day);   lcd_print("/");
    lcd_print_uint16(t_init.month); lcd_print("/");
    lcd_print_uint16(t_init.year);

    lcd_set_cursor(1, 0);
    lcd_print_uint16(t_init.hour);   lcd_print(":");
    lcd_print_uint16(t_init.minute); lcd_print(":");
    lcd_print_uint16(t_init.second);
    _delay_ms(4000);


    while(1){
        uint16_t elapsed = 0;

        while (elapsed < PAGE_HOLD_MS) {
            // sensor okuma
            BME280_Data bme = bme280_read();
            uint16_t ldr = adc_read(ADC_LDR);
            uint16_t mq  = adc_read(ADC_MQ135);
            DS1307_Time t = ds1307_get_time();

            // lcd guncelleme
            lcd_show_page(current_page, &bme, ldr, mq, &t);
            update_status(&bme, mq);

            // tekrar guncelleme (gercek zamanli)
            _delay_ms(PAGE_REFRESH_MS);
            elapsed += PAGE_REFRESH_MS;
        }

        // sayfa degistirme
        current_page++;
        if (current_page >= PAGE_COUNT) {
            current_page = 0;
        }

        // yaklasik 16 saniyede bir gonderim
        send_counter++;
        if (send_counter >= 4) {
            BME280_Data bme = bme280_read();
            uint16_t ldr = adc_read(ADC_LDR);
            uint16_t mq  = adc_read(ADC_MQ135);

            esp32_send_data(
            (uint16_t)bme.temperature,
```

```
                (uint16_t)bme.humidity,
                (uint16_t)bme.pressure,
                ldr,
                mq
                );
                send_counter = 0;
            }
        }
}
```

9. KAYNAKÇA

[1] Bosch Sensortec, "BME280 Çevresel Sensör — Veri Sayfası," BST-BME280-DS002, Rev. 1.9, 2023. https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme280-ds002.pdf

[2] Maxim Integrated, "DS1307 64 x 8 Seri Gerçek Zamanlı Saat," Veri Sayfası, Rev. 3, 2015. https://datasheets.maximintegrated.com/en/ds/DS1307.pdf

[3] Espressif Systems, "ESP8266EX Veri Sayfası," Sürüm 6.7, 2023. https://www.espressif.com/sites/default/files/documentation/0a-esp8266ex_datasheet_en.pdf

[4] Atmel Corporation, "ATmega328P Veri Sayfası," Belge No. 8271D, 2018. https://ww1.microchip.com/downloads/en/DeviceDoc/ATmega48A-PA-88A-PA-168A-PA-328-P-DS-DS40002061B.pdf

[5] MathWorks / ThingSpeak, "ThingSpeak Dokümantasyonu," https://www.mathworks.com/help/thingspeak/

[6] Seeed Studio, "WiFi Seri Alıcı-Verici Modülü (ESP8266)," https://wiki.seeedstudio.com/WiFi_Serial_Transceiver_Module/

[7] Espressif Systems, "ESP32-WROOM-32D Veri Sayfası," 2023. https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf

[8] Görsel İşitsel Teknoloji, "Arduino Wifi - Esp8266 - Thingspeak veri çekme," YouTube. https://www.youtube.com/watch?v=EOA0BXmEcvs

[9] Görsel İşitsel Teknoloji, "Arduino Wifi - Esp8266 - Thingspeak veri gönderme," YouTube. https://www.youtube.com/watch?v=HvbQHnS86CA

[10] Espressif Systems Arduino Core, "ESP32 Arduino WiFi.h / HTTPClient.h," esp32 Arduino kütüphanesi, 2024. https://github.com/espressif/arduino-esp32